

Project Report

Project Goal: understand performance

- ▶ Give a description of your problem.
- ▶ Describe performance analysis, including:
 - ▶ Serial tuning and reorganizations
 - ▶ Strong and weak scaling experiment (speedup plots)
 - ▶ Profiling of communication and computation
 - ▶ Tuning of parallelism (communication, synchronization, etc)
 - ▶ Comparison to analytical models
 - ▶ Comparisons between alternate organizations
- ▶ Describe how this work might continue given more time.
- ▶ Provide comments in the code.

Goals:

- ▶ Reason about code performance
 - ▶ Many factors: hardware, software, algorithms
 - ▶ Want simple, “good enough” models
- ▶ Read/judge HPC literature
- ▶ Apply model numerical HPC patterns
- ▶ Tune existing codes for modern HW
- ▶ Apply good software practices

These things matter:

- ▶ ILP: Pipelining, concurrent execution, and vectorization
- ▶ Memory hierarchy and the cost of cache misses
- ▶ Communication costs (latency and bandwidth)
- ▶ Synchronization overheads

Numerical ideas and improving performance:

... thinking about high-performance numerics often involves:

- ▶ Tiling and blocking algorithms; building atop the BLAS
 - ▶ Ideas of sparsity and locality
 - ▶ Graph partitioning and communication / computation ratios
 - ▶ Information propagation, deferred communication, ghost cells
 - ▶ Big picture view of sparse and direct iterative solvers
 - ▶ Some multilevel ideas
 - ▶ And a few other numerical methods (FMM, MC, MD) and associated programming patterns
- ▶ Zeroth steps
 - ▶ Working code (and test cases) first
 - ▶ Be smart about trading your time for CPU time!
 - ▶ First steps
 - ▶ Use good compilers (if you have access – Intel is good)
 - ▶ Use flags intelligently (`-O3`, maybe others)
 - ▶ Use libraries someone else has tuned!
 - ▶ Second steps
 - ▶ Use a profiler
 - ▶ Learn some timing routines (system-dependent)
 - ▶ Find the bottleneck!
 - ▶ Third steps
 - ▶ Tune the data layout (and algorithms) for cache locality
 - ▶ Put in context of computer architecture
 - ▶ Now tune
 - ▶ Maybe with some automation (Spiral, FLAME, ATLAS, OSKI)

Parallel environments:

- ▶ MPI
 - ▶ Portable to many implementations
 - ▶ Giant legacy code base
 - ▶ Does keep evolving (e.g. RDMA support)
- ▶ OpenMP
 - ▶ Parallelize C, Fortran codes with simple changes
 - ▶ ... but may need more invasive changes to go fast
- ▶ CUDA, OpenCL, etc
 - ▶ Highly data-parallel kernels (e.g. for GPU)

Libraries:

- ▶ Dense LA: LAPACK and BLAS (ATLAS, Goto, Veclib, MKL, AMD Performance Library)
- ▶ Sparse direct: Elemental, Pardiso (in MKL), UMFPACK (in MATLAB), WSMP, SuperLU, TAUCS, DSCPACK, MUMPS,
...
- ▶ FFTs: FFTW
- ▶ Graph partitioning: METIS, ParMETIS, SCOTCH, Zoltan, ...
- ▶ Other; deal.ii (FEM), SUNDIALS (ODEs/DAEs), SLICOT (control), Triangle (meshing), ...
- ▶ Frameworks: PETSc/Trilinos
 - ▶ Gigantic, a pain to compile... but does a lot
 - ▶ Good starting places for ideas, library bindings!
- ▶ Collections: Netlib (classic numerical software), ACTS (reviews of parallel code)
- ▶ MATLAB, Anaconda Python distro, etc. add value in part by selecting and pre-building interoperable libraries

Scripting:

... because we don't want to spend all our lives debugging C memory errors, it helps to make judicious use of other languages:

- ▶ Many options: Python, Ruby, Lua, Julia, R, ...
- ▶ Wrappers help: SWIG, tolua, Boost/Python, Cython, etc.
- ▶ Scripts are great for
 - ▶ Prototyping
 - ▶ Problem setup
 - ▶ High-level logic
 - ▶ User interfaces
 - ▶ Testing frameworks
 - ▶ Program generation tasks
 - ▶ ...
- ▶ Worry about performance at the bottlenecks!

Development ideas:

- ▶ Still lots of “traditional” HPC computations
 - ▶ Large-scale optimization
 - ▶ PDE solves
 - ▶ Engineering simulation
- ▶ Graph applications?
 - ▶ Different properties from PDEs
 - ▶ Similar applications
- ▶ Also lots of stats / ML computations
 - ▶ Often more opportunities for parallelism
 - ▶ Often more data, less accuracy – I/O becomes the key
 - ▶ Lots of work on frameworks for these problems
 - ▶ Closer to traditional HPC over time...
- ▶ “Big data” and DB ideas
 - ▶ Lots of relatively modest computations over lots of data
 - ▶ Still rather different community from lots of HPC